

Z-Ant: Static Memory Planning and Compiler-Level Contributions to an Open-Source Embedded Inference Framework

Treating neural-network deployment as a compilation problem to emit dependency-free static libraries for resource-constrained hardware.

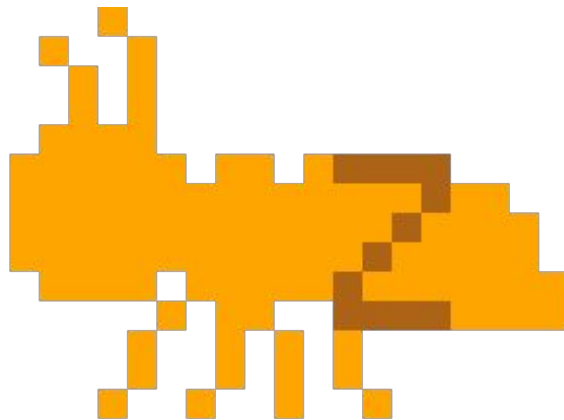
Contributions

Positional error-return for inference, ONNX [split](#) and [abs](#) operations, static memory planning system with branch-and-bound planning and configurable heuristics, new CI/CD with improved coverage and better infrastructure, integration of CMSIS-NN

Authors & Institution

Matteo Roda and Elia Sandrini

Politecnico di Milano, B.Sc. Engineering of Computing Systems



Z-Ant Replaces Unpredictable Runtime Execution with Deterministic Compilation

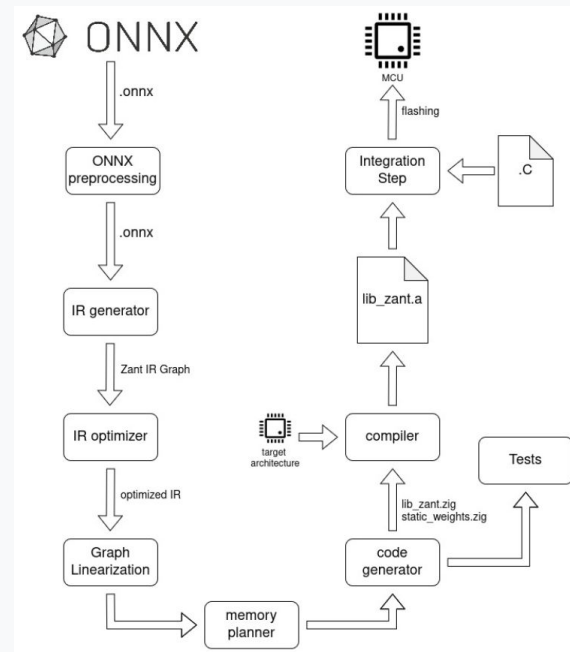
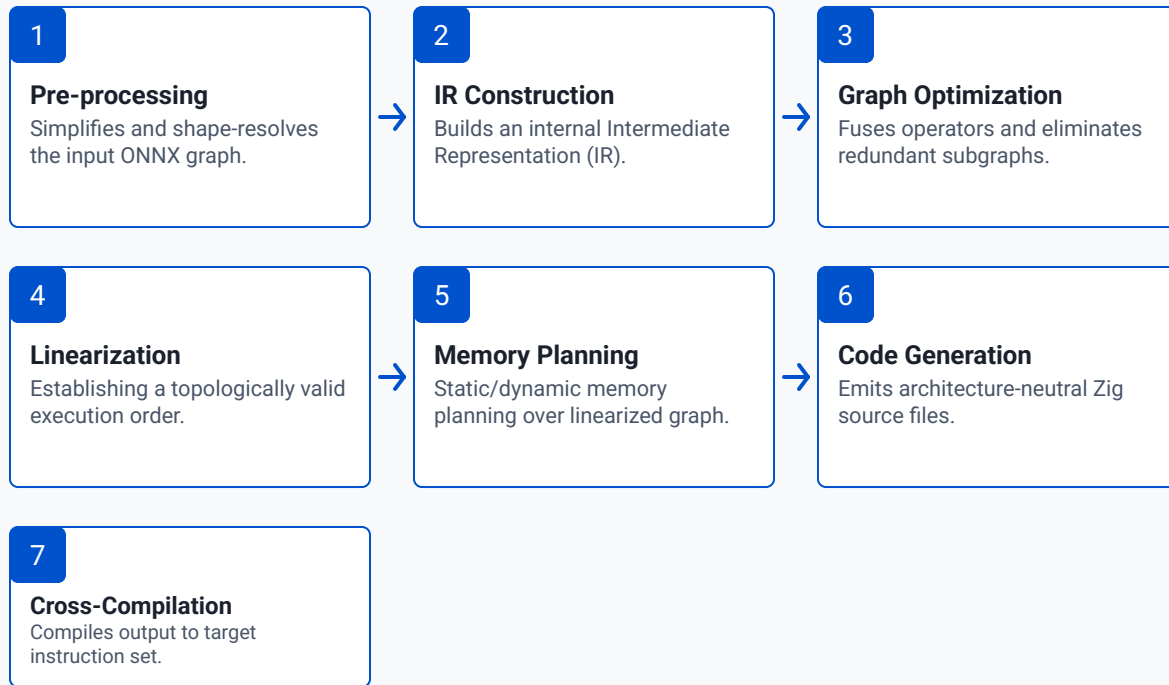
Traditional Inference Frameworks

- Designed around dynamic memory allocation
- Rely on runtime graph traversal
- Depend on underlying operating system services
- Produce binaries that are too large and unpredictable for bare-metal firmware

The Z-Ant Approach

- Treats deployment as a compilation problem
- Resolves all scheduling, memory planning, and optimization decisions at compile time
- Produces a deterministic static library written in Zig and C
- Ready for direct integration into embedded firmware without external dependencies

The Seven-Stage Compilation Pipeline Translates ONNX to Bare-Metal Executables



Structured Error Reporting and Operator Alignment

Positional Error-Reporting Scheme

- Replaces uniform `-1` failure with operator-indexed return codes to enable precise fault localization
- Failure `-101` identifies the first operator, `-102` the second, and so on
- Range `[-1, -99]` reserved for system-level errors
- Implemented via `getMathErrorReturn` utility to maintain consistency across the codegen layer
- All `write_op` functions updated uniformly

All operators support dual execution paths: **normal** (self-allocating) and **lean** (preallocated).

split type & memory realignment

- `[]const usize → [] const i64` (ONNX spec alignment)
- `split_lean` redefined: validates shapes and copies, no allocation

abs new operator implementation

- ONNX element-wise absolute value via Zig `@abs` intrinsic
- Both lean and normal execution paths implemented
- Validated against ONNX Runtime reference output

Static Memory Planning: Motivation and v0 Baseline

- **The Constraint:** Dynamic allocation of intermediate tensors on microcontrollers risks memory fragmentation and fatal out-of-memory crashes during inference.
- **The Strategy:** Buffer reuse. Intermediate tensors must map to statically sized backing buffers fixed at code-generation time. Buffers are reused across tensors whose active windows do not overlap.
- **The Redesign:** Z-Ant introduces a redesigned memory planning subsystem featuring a configurable offline interval-packing heuristic and an exact branch-and-bound planner.
- **Objective:** minimize total element-count across typed backing buffers. NP-hard in general (typed interval scheduling + bin-packing).

v0: Online Greedy Allocator

- Operates with a purely local view during graph traversal.
- Uses an online free-list and reference counting.
- Misses global reuse opportunities because it only sees buffers whose previous tenants have already been reclaimed.

v1: Offline Interval-Packing Heuristic

Step 1

Collect Lifetimes

Scan graph: compute $b(t)$ and $d(t)$ for all tensors

Step 2

Sort Tensors

Select ordering via `-Dstatic_planning` flag

Step 3

Best-Fit Placement

Assign to smallest compatible existing typed buffer

Six Tensor Ordering Strategies

size_first: Largest tensors first; most constrained slots secured early

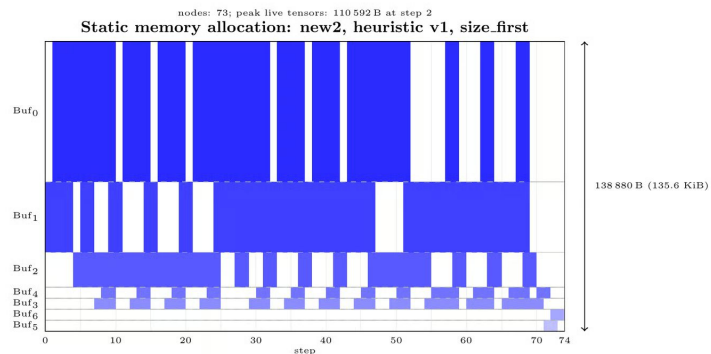
pressure_then_size: $\text{size} \times \text{liveness}$ primary; size breaks ties

pressure_then_liveness: $\text{size} \times \text{liveness}$ primary; liveness breaks ties

liveness_first: Longest-lived first; can misfire on large short activations

first_step: Production order only; lower-bound comparison baseline

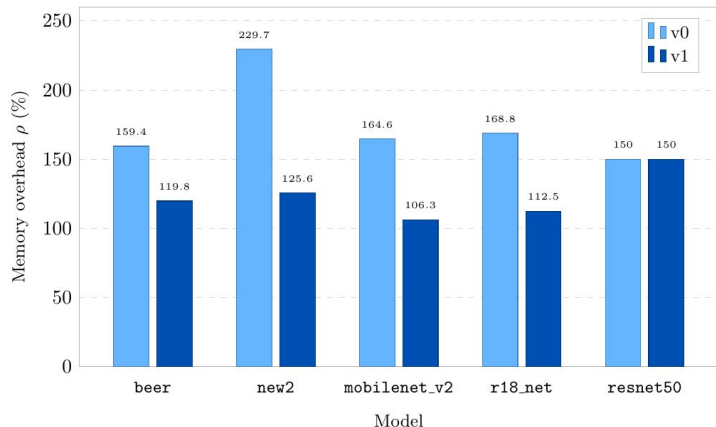
+ **_inverse_first_step:** Suffix: reverses tie-breaker for later-produced tensors



Static memory allocation for `new2` with the `size_first` strategy. The stacked bands are planner backing buffers over execution steps: buffers 0-4 store u8 quantized activations, while buffers 5-6 store f32 output-path tensors. Shaded regions show tensor lifetimes; white regions are allocated but idle space

Redesigned Planning Reduces Allocation Overhead by up to 45 Percentage Points

The v1 planner reduced allocation overhead by 24.8 to 45.3 percentage points across four of five benchmarked models.



Memory overhead ρ per model (v0 vs. v1). Bars show the best (lowest) ρ achieved across all allocation strategies. $\rho = 100\%$ means theoretical minimum.

Peak Efficiency: MobileNetV2

106.3%

Approached theoretical minimum memory footprint.

The regular inverted-residual structure concentrates peak pressure on just two non-overlapping tensors — `size_first` detects this immediately.

Best Reduction: new2

-45.3%

Comparing v1 to v0 heuristic planners

Quantized concat-heavy graphs fragment at branch joins — v1's exact lifetime tracking collapses recurring size tiers into reuse buckets.

Structural Limits: ResNet50 150.0% Overhead

The Skip Connection Bottleneck

Skip connections force large early feature maps to remain live until convergence. Peak-pressure tensors saturate the allocation floor regardless of strategy.

Worked Example: Heuristic v1 Compared to v0

Setup: 8 tensors share the same element type and use inclusive lifetimes.

- A, B, C, D: Size 3; Op0 produces A/B, Op1 reads A -> C, Op2 reads B -> D, Op3 reads C/D.
- X, Y: Size 4; Lifetimes [4,6] and [6,8].
- M, L: Sizes 6 & 10; Overlap at step 10.

v0 Result: 36 elements

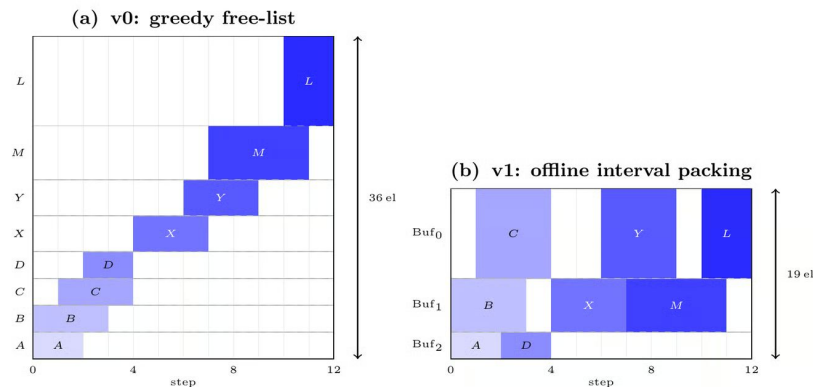
Online greedy allocation creates one slot per tensor in this example.

It misses A/D reuse because conservative child-borrow tracking keeps A locked when D is allocated.

v1 Result: 19 elements

Offline planning reaches the lower bound: one size-10 buffer, one size-6 buffer, and one size-3 buffer.

It combines exact last-use intervals, size_first ordering, and best-fit placement.



Memory allocation comparison. v0 allocates 36 elements because its child-borrow approximation misses the A/D reuse and later tensors require separate slots. v1 reduces the allocation to 19 elements by using exact last-use intervals, size_first ordering, and best-fit buffer placement.

Exact Planning & Buffer Zeroing Correctness Fix

Branch-and-Bound Exact Planner

- **Triggered** when linearized graph ≤ 25 nodes.
- **Ordering:** `size_first` ensures most constrained tensors are placed first, allowing pruning to fire earlier.
- **Pruning:** Single criterion — abandon branch when partial allocation > current best.
- **Complexity:** $O(T!)$ worst case, but practical within the 25-node bound.
- `-Dforce_bnb` overrides threshold.

Buffer Zeroing on Reuse

- **Silent Hazard:** Accumulation operators inherit residual values from a buffer's previous tenant.
- **Fix:** Emit `@memset(tensor_x.data, 0)` before each op in the generated `predict()` body.
- **Optimization:** Suppressed for in-place operators via Zig `for...else` pattern.
- **Cost:** Paid once per inference call — no compile-time overhead.

Revitalizing the CI/CD Pipeline Restores Automated Reliability

Prior State

The infrastructure was not updated for seven months. Then, the module restructuring rendered previous workflow files non-functional.

The duplicated code required changes in every job to fix deprecated functionalities.



```
.github
├── ISSUE_TEMPLATE
├── actions
│   ├── setup-python
│   │   └── action.yml
│   └── setup-zig
│       └── action.yml
└── workflows
    ├── zig-benchmarks.yml
    ├── zig-codegen-tests.yml
    └── zig-tests.yml
```

```
- name: Install Python dependencies
  shell: bash
  run: |
    python -m pip install --upgrade pip
    pip install -r tests/CodeGen/Python-ONNX/requirements.txt

- name: Set up Zig
  uses: goto-bus-stop/setup-zig@v2
  with:
    version: ${ inputs.version }
```

01

Consolidation

Duplicated setup logic merged into reusable GitHub composite actions (setup-zig, setup-python).



02

Accessibility

workflow_dispatch trigger added for on-demand execution via web interface without branch pushes.



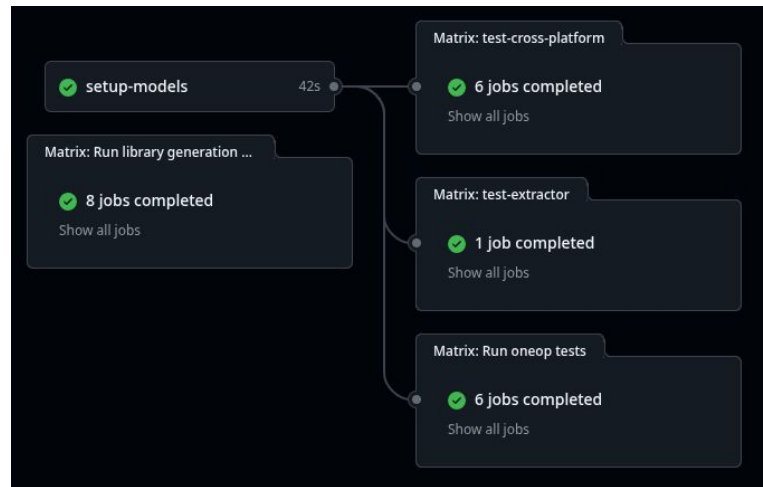
03

Maintenance

Corrected filesystem paths and deprecated CLI references to match new codebase layout.

Matrix-Based Multi-Flag Testing Doubles CI/CD Coverage

- **Cross-Platform Expansion:** The compilation job now uses a matrix strategy to test three target triples (x86_64-linux, aarch64-linux, x86_64-windows) across two optimization levels (Debug, ReleaseFast), yielding six configurations.
- **Library Generation:** The code-generation job tests combinations of three independent boolean axes, producing eight unique library-build configurations per model.
- **Robust Diagnostics:** The fail-fast option is explicitly disabled across all matrix jobs. Because matrix cells are independent, this ensures a single failure does not abort the remaining tests, preserving critical diagnostic output.



Workflow run results

Work in Progress: CMSIS-NN Integration for Hardware Acceleration

The Target: CMSIS-NN

ARM's hand-optimized library for Cortex-M, utilizing SIMD, DSP, and Helium vector extensions.

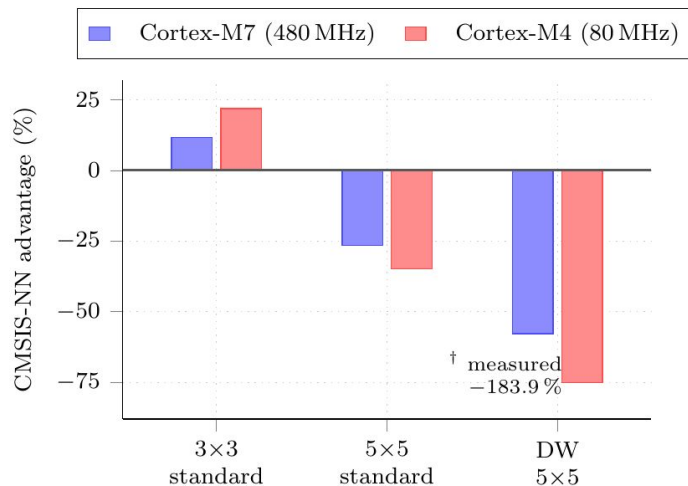
Implementation: Selective Backend

Dispatching to CMSIS-NN only when the hardware supports CMSIS-NN.

Implementation: Wide Kernel Acceleration

Previous implementation showed performance gains only for some kernels: the goal is finding the cause and reaching full CMSIS-NN support for convolutional networks.

Goal: Accelerate Quantized Convolutions (QLinearConv)



Previous implementation benchmarks
Best Percentage Gain per Model (v0 → v1)